后训练学习计划 • 第 15 周（12/14-12/20 • Phase 4 面试火力准备 • 第三周）

本周定位：**把项目讲成能被追问五层的东西，并把简历真的送出去**。刷题继续推进到累计 ≥85 题，手撕三件全部换成"对齐/偏好/参数高效"这三块后训练的必考地，第 3 次模拟面按**对镜录屏**执行。**本周不训练**，卡时上限压到 2 小时，只用于把失败案例里那几条"只有结论没有证据"的补跑一次。 用法说明：每天先看「今日总览」，再从任务 1 逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗 = 硬性要求**，灰色小字块 = 注释（可跳过，不影响主线）。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一	DP（上）6 题 + M2 讲述版初稿 + 投递第一批 ≥8 家	talk-m2 六段成稿、台账新增 8 行	6h
Day 2 周二	DP（下）6 题 + 手撕 1：Bradley-Terry RM 损失 + M2 五层下挖（L1-L3）	`BT TESTS OK`、M2 追问 ≥9 问	6.5h
Day 3 周三	贪心 6 题 + M3 讲述版 + 八股漏洞闭卷回补	talk-m3 成稿、闭卷 ≥10/14	6h
Day 4 周四	图/层序 6 题 + 手撕 2：GRPO 组内优势 + M3 五层下挖（L4-L5）+ 开卡补跑	`GRPO ADV OK`、补跑证据入库	6.5h
Day 5 周五	字符串 6 题 + 手撕 3：LoRA merge 与回滚 + M4 讲述版与下挖	`MERGE TESTS OK`、talk-m4 成稿	6h
Day 6 周六	回炉 + **第 3 次模拟面（对镜录屏）** + 三面讲稿定稿	mock-03 录屏 ≥85 分钟、复盘 ≥12 条	6.5h
Day 7 周日	补漏 + 闭卷复测 + 总验收 + Week16 预习	验收 ≥6/7 项通过	4.5h

注：**本周卡时上限 2h，预估花费 ¥3-5**（4090 按量 ¥1.5-2.5/时，2 小时顶格）。允许开卡的唯一理由：`fail-log.md` 里有条目卡在"只有结论没有证据"（例如 M2 的 rank 消融只留下结论没留表、M3 的 reward 曲线截图丢了），Week 14 Day 5 已经写好 `rerun.sh`，本周执行它。**规则：开卡前把 `rerun.sh` 与"这条补跑要支撑哪个简历/fail-log 条目"一起贴出来，写不出对应条目就不开**。超了就把那两条的数值退回"未量化"——面试里说"当时没留存记录"不丢人，编一个精确值才致命。

注：本周最大风险是**讲述版写成 README**。README 是给自己看的，讲述稿是给一个不打算听完的人听的。判据很硬：15 分钟版本必须能在 13-16 分钟内讲完，且第 5 层追问（"数据量 ×10 时哪个环节先崩"）答得上来。第二大风险是**投递又往后推**——本周 ≥20 家是硬指标，因为 2027 年 2 月底-3 月暑期提前批开抢前，你至少要有一轮真实反馈来校准简历。

注：砍单顺序（实在做不完时按这个顺序砍）：Day 3 任务 3 八股回补 → Day 5 任务 4 台账跟进 → Day 4 任务 4 开卡补跑（直接退回"未量化"）→ Day 2/Day 4 的"补名额"新题 → **刷题、三份讲述稿 + 五层下挖、集中投递、第 3 次模拟面四项不许砍**。这四项分别是面试的四个必查面，砍一项就缺一块。

二、开始前：目录 / 台账归位 / 讲述与追问规范（约 45 分钟，前一晚做完）

1. 建目录（bash 全用正斜杠，反斜杠会被吞）：

```
mkdir -p E:/study/posttrain/week15/{day1,day2,day3,day4,day5,day6,day7}
mkdir -p E:/study/posttrain/llm-posttraining-lab/interview
```

2. 本机环境沿用 Week 14 那个 venv（本周三件手撕全在 CPU 上跑，**不需要 GPU**）：

```
cd E:/study/posttrain
source .venv/Scripts/activate          # PowerShell:  .venv\Scripts\Activate.ps1
python -c "import numpy, torch; print(numpy.__version__, torch.__version__)"
```

> 注：如果你换回了 conda 环境，`import torch` 抛 `OMP: Error #15: ... already initialized`（conda 的 MKL 与 torch 自带 OpenMP 抢同一运行时）。正解仍是那个独立 venv；急着往下做就先 `export KMP_DUPLICATE_LIB_OK=TRUE`（Git Bash）/ `set KMP_DUPLICATE_LIB_OK=TRUE`（cmd）绕过。Day 2 与 Day 5 两件都要 `import torch`，别在错误的环境里跟它对赌时间。

3. **台账归位**（本周起固定，别再两边记）：
   - 对外资产放仓库 `llm-posttraining-lab/interview/`：`solved.md`（刷题累计口径）、`fail-log.md`（Week 14 定稿的失败案例清单）、`bagu-2.md`、本周新增 `talk-m2.md` / `talk-m3.md` / `talk-m4.md` / `drill.md`。**这里只放能拿去给面试官看的东西。**
   - 私密记录放 `E:/study/posttrain/jobhunt/`：`applications.csv`（投递台账，含内推码与联系方式）、`leetcode/log.md`（逐题用时与提交次数）。**这一份不要 push 到公开仓库。**
   - 一致性校验：`solved.md` 的累计行数是对外口径，`log.md` 是明细，Day 6 用 `wc -l` 对一次，差超过 3 行就把明细补齐。
4. 把上周的三样东西拉到今天能看见的位置：`week14/gaps.md`（八股闭卷错的题与未 AC 题）、`fail-log.md` 的"前 6 条索引"、Week 14 Day 5 写的 `rerun.sh`。今天 Day 1 任务 2 直接要用。

写作规范：六段讲述结构 + 五层下挖阶梯（读 15 分钟，本周所有讲稿按这个写）

1. **六段结构**（每个项目的 `talk-*.md` 都按这六段写，段落标题原样用）：
   - 动机：为什么做这件事，它在真实产品里解决什么（**不许从技术名词起手**）
   - 方案：阶段怎么分、每阶段为什么存在、我用了哪条管线
   - 数据：来源、规模、清洗与去污染、许可与脱敏
   - 指标：跟谁比、怎么测、评测集怎么定 rubric、一致率
   - 失败尝试：2-3 条，每条一句话现象 + 一句根因（从 `fail-log.md` 里挑，别现编）
   - 改进：现在再做一次会改哪两处，以及为什么当时没改（时间/显存/认知，说实话）
2. **五层下挖阶梯**（`drill.md` 每个项目按层写，每层 ≥3 问，三个项目合计 ≥45 问）：
   - L1 事实层：这个数字/配置/模块具体是什么，记在哪个文件
   - L2 选择层：为什么是这个选择，替代方案是什么，你怎么排除的
   - L3 权衡层：这个选择的代价；什么条件下结论会反过来

- L4 失败层：这条路上真出过什么问题，你怎么定位的，有没有没解决的
- L5 扩展层：数据 ×10 / 模型从 1.5B 到 7B / 上线到真实并发，**哪个先崩**，为什么

3. 每问四段写全：**问 / 我的 60 秒答（口播稿，不是要点）/ 证据出处（文件或截图路径）/ 漏洞标记**（`已验证` / `量级估计` / `没做过`）。**"没做过"这一栏必须存在且不许全填"已验证"**——面试官一句"这个你测过？"就能把假答案打成零分，而"我没测，但我推测是 X，测法是 Y"是加分。

三道经典追问：差答案 vs 好答案对照（照这个结构重写你自己的版本）

Q1 "LoRA 的 rank 为什么选 16？" - 差答案：① "常见值/默认值"（等于没答）② "越大效果越好，显存不够"（一句都对但没信息）③ 只报一个数字，说不出替代方案。 - 好答案要含五件东西： 1. 先说 r 到底决定什么：ΔW = BA 的秩，可训参数量随 r 线性（单层约 `r*(d_in+d_out)`），容量与 adapter 体积一起涨。 2. 我的对照：`r=4 / 8 / 16 / 32` 同一份数据、同一 lr、同一 target_modules，指标差多少（写量级 + 出处；没留表就写"未量化"并说明当时为什么没留）。 3. 为什么不选小的：r=4 时具体哪个能力没学到（举一个输出层面的现象，不是"效果差一点"）。 4. 为什么不选大的：r=32 增益不到一个可感知的量级，但 adapter 体积翻倍、更容易过拟合偏好数据。 5. 收尾的加分句：换 r 时我锁的是 `scaling=alpha/r` 这个比值，不锁它扫出来的差异里混着有效学习率的效应——**这一句是区分"用过 LoRA"和"懂 LoRA"的分水岭**。 - 第 5 层追问："7B 上还锁 alpha=32 吗？"答：r 与 alpha 要重扫，因为基座容量变大后旁路更新的相对占比变了；我的做法是先固定 `alpha/r=2` 再扫 r（这是量级判断，我没做完整消融，会诚实说出来）。

Q2 "reward hacking 出现过吗？" - 差答案：① "没有，我 KL 控制得比较好"（做过 RL 的人不会这么说，等于自曝）② "有，输出变长了"——没有判据、没有修法、没有遗留。 - 好答案的五步： 1. 一句确认："出现过，是格式项被吃掉的版本。" 2. 我怎么发现：reward / 平均输出长度 / KL 三条画在同一张图上，看到"reward 涨但正确率平、长度同向上"。 3. 判据（要能背）：**reward 与长度同向上升、reward 与可验证正确率脱钩 = hacking 信号**；只看 reward 曲线一定会被骗。 4. 我做了什么：把 reward 拆子项归因 → 格式校验收紧到"只认最后一个答案" → 加长度惩罚 → KL 系数拉大一档 → 必要时重训/重校 RM。 5. 遗留：judge 与人工盲评的一致率我现在还不满意，量级大概是多少、当时只抽了 N 条人工复核。 - 抽象收口一句（背下来）：**"hacking 不是模型坏了，是我的奖励函数写漏了——RM 分布外的区域，策略一定会找到。"**

Q3 "KL 系数怎么调的？" - 差答案：① "用的 0.2"（报数不讲法，第二层追问就崩）② 把 PPO 的 KL 惩罚系数和 DPO 的 β 混成一谈还不声明口径。 - 好答案的六步： 1. **先定术语与位置**：PPO 里 KL 是逐 token 加在 reward 端的惩罚（系数常记 β/KL-coef）；DPO 里的 `beta`（TRL 默认量级 0.1）是偏好损失里偏离参考策略的强度；GRPO 的常见实现是 loss 端的 `kl_coef`。**三个不是同一个旋钮**，先说清自己在讲哪一个——这一句说出来，面试官就知道你真配过。 2. 我盯什么信号：健康 = reward 上行 + KL 在受控区间小幅波动；病态 = KL 单调上涨且 reward 不涨，或 KL 几乎不动而 reward 猛涨（学不动或在钻空子）。 3. 我的取值量级与起点：从一个小值起步往上调，一次只动一档，其余锁死。 4. 代价双向：调太大 → 学得慢、贴回基座、能力上限被压；调太小 → 语言风格漂移、复读、更易 hacking。 5. 可执行纪律（这句最值钱）："我把 **KL 连续 N step 单调上升且 reward 不涨** 设成自动停训条件"——把玄学变成开关。 6. 诚实遗留：我只做了两档对照，没做系统扫参；所以我讲不了最优值，只能讲我为什么不用极端值。

> 注：上面三题的"好答案"是同一个模板：**定位术语 → 我盯的信号 → 我的量级 → 双向代价 → 遗留问题**。任何一层缺了都会被判成"背过没做过"。本周 `drill.md` 里 45 问全部按这个五段结构写，写完自查一遍：有哪几问只有前两层，那几问就是你面试时的出血点。

Day 1 • 周一：DP（上）6 题 + M2 讲述版初稿 + 投递第一批

今日总览（6h）：DP 从零开始建表直觉；把 M2 的 15 分钟讲述版**一次写完整**（初稿允许烂，不许短）；晚上投出第一批 ≥8 家，投递不许往后推。

任务 1：DP（上）6 题（约 150 分钟）

1. 题单（每题限时写进台账）：**70 爬楼梯(15) / 118 杨辉三角(15) / 198 打家劫舍(25) / 279 完全平方数(25) / 322 零钱兑换(30) / 139 单词拆分(40)**。
2. 写代码前先在本子上写三行：**状态定义 / 转移方程 / 边界与填表顺序**，写完才允许碰键盘。三行写不出就算未 AC。
3. `322 零钱兑换` 要能说出为什么贪心不行（coin=[1,3,4] 凑 6，贪心给 4+1+1=3 枚，最优是 3+3=2 枚）——这是面试高频反例，必须自己背下来。`139 单词拆分` 用 `set` 存词表，内层枚举切分点，说清复杂度 $O(n^2 \cdot L)$。

完成判据：6 行新题台账（`solved.md` + `jobhunt/leetcode/log.md` 两边同步），**独立 AC ≥5**；每题的"状态定义"三行有留痕（拍照或粘进 log 备注列）；DP 类第一组计数进台账。

> 注：DP 是刷题里最容易"看题解秒懂、自己写崩"的一类，因为题解给的是转移方程而不是**怎么想出转移方程**。所以今天的判据把"三行先行"卡死：状态定义写歪了，后面填表全白干。超时 40 分钟按 Week 13 的规矩处理——看题解只看到思路、第二天重做才计数。

任务 2：M2 讲述版 15 分钟初稿（约 110 分钟）

1. 打开 `interview/talk-m2.md`（Week 14 Day 7 只写了六段小标题），今天填内容。结构：动机 / 方案 / 数据 / 指标 / 失败尝试 / 改进，每段前标目标秒数（合计 900 秒）：**动机 120 / 方案 240 / 数据 180 / 指标 180 / 失败尝试 120 / 改进 60**。
2. 素材从三处搬：`m2-sft/README.md`（配置与数据规模）、`fail-log.md` 的 M2 段（挑 2-3 条，只搬"现象 + 根因"各一句）、Week 6/8 的评测表（held-out 偏好对准确率、20 题盲评）。**每个数字后面标出处路径**。
3. 写完朗读计时两遍：第一遍纯朗读记时长；第二遍按 13-16 分钟目标删段。**超过 17 分钟就砍"方案"段的技术名词罗列，只留"为什么分这个 Stage"**。

完成判据：`talk-m2.md` 六段齐全且每段有目标秒数；朗读时长记录在文件头，落在 **13-16 分钟**；文中数字 100% 带出处或标了"未量化"。

> 注：M2 最容易讲死的地方是"我用了 LoRA 和 DPO"起手。正确起手是产品事实："我的产品要给用户生成记忆口诀，线上只有云端 API 这一条路，延迟和成本都卡在这。"技术是实现手段，**面试官买的是"你为什么要做"而不是"你用什么"**。

任务 3：投递第一批 ≥8 家（约 80 分钟）

1. 从 `jobhunt/targets.md` 的三档清单里挑第一批 8 家：**业务侧 ≥4 + 模型公司 ≥3 + 大厂实验室 ≥1**，优先月之暗面 / 智谱 / 美团这类后训练方向明确、对无论文友好的岗位（这是你手里资料给的判断，别一上来全砸

最难的档）。

2. 每家三件事：① 按 Week 14 台账里"我这份简历投它时该改哪一句"那栏改简历首屏（**只改首屏那一句，不许开第 8 版**）；② 3 行自述，第一句是 200 字叙事压缩版（真实产品 → 日志造数据 → SFT/DPO/GRPO → 自建评测 + A/B → 小模型换掉 API 的成本测算）；③ 登记进 `applications.csv`，九个字段一个不许空，拿不到写 `N/A`。
3. 内推：今天至少走 3 家内推渠道（牛客/即刻/小红书员工内推码，或认识的在职的人），**内推码记进台账**，后面催进度要用的就是这一栏。

完成判据：`applications.csv` 今天新增 **≥8 行有投递日**（含 Week 13 首批即累计 ≥16 行）；每家写了"内推人/内推码"或 `N/A`；改过的首屏句每家留了一份文本（后面复投还要用）。

> 注：投递的反馈周期是 3-10 天，本周投出去的正好是下周面试周的弹药。今天不投，Week 16 就没有"已约面"这个状态可以复盘。**"再准备一下"是求职周最贵的四个字的**。

任务 4：收工与明天预排（约 20 分钟）

1. `week15/review.md` 开个头：今天实际用时、投递新增数、卡时（今天应为 0）。
2. NOTES.md 三行：`322` / `139` 哪道没独立 AC、M2 讲述里自己最不放心的是哪一段、明天 BT 损失预计耗时。

完成判据：review.md 有今天的三行账；`solved.md` 顶部累计数更新（应为 **≥61**：Week 14 末 55 + 今天 6）。

今日验收：DP（上）6 行（独立 AC ≥5）+ `talk-m2.md` 六段成稿且朗读 13-16 分钟 + 台账新增 ≥8 行。

Day 2 • 周二：DP（下）6 题 + 手撕 BT/RM 损失 + M2 下挖 L1-L3

今日总览（6.5h）：DP 第二组 + 手撕第 1 件（把 Bradley-Terry 损失写对并自证）。BT 损失是 RM 与 DPO 的共同地基，判据不是"写出来"，是**梯度方向与对称性都测过**。

任务 1：DP（下）6 题（约 150 分钟）

1. 题单：**300 最长递增子序列(30) / 152 乘积最大子数组(25) / 416 分割等和子集(30) / 62 不同路径(20) / 64 最小路径和(20) / 1143 最长公共子序列(25)**。
2. `300` 两种写法都要：O(n²) DP 与 O(n log n) 贪心 + 二分（`tails` 数组），**第二种是高频追问**，答不出二分的不变量（tails[k] = 长度为 k+1 的递增子序列的最小末尾）等于没写。
3. `152 乘积最大子数组` 的坑在负数：必须同时维护 max 与 min，讲清为什么。`416` 是 0-1 背包判可行性，一维滚动数组内层**倒序**，正序会重复取同一件。

完成判据：6 行新题台账，**独立 AC ≥5**；`300` 的两道写法都能闭卷敲出；`416` 的一维倒序能一句话说清原因（避免同一物品在本轮被重复使用）。

> 注：`1143 LCS` 在后训练面试里有个真实映射：偏好数据去重与去污染时用的 n-gram 交集、以及"两条样本是不是同一条改写"，本质都是序列相似度。被问到去污染为什么用 13-gram 交集而不是编辑距离时，这个类比能救你——**因为 n-gram 求交集是 O(n) 哈希，序列比对是 O(n•m)**，而你要扫的是几十万条。

任务 2：手撕 1 • Bradley-Terry / RM 损失（约 110 分钟）

1. 新建 `week15/day2/bt_rm.py`：

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ScalarRM(nn.Module):
    """极简 RM：序列特征 -> 线性回归头 -> 标量分数。
    真实 RM 是 基座 + pool/末 token -> Linear(1)，这里只写损失相关部分。"""
    def __init__(self, dim: int):
        super().__init__()
        self.head = nn.Linear(dim, 1)

    def forward(self, feats):                  # feats: (B, dim)
        return self.head(feats).squeeze(-1)    # -> (B,)

def bt_loss(r_chosen, r_rejected, label_smoothing: float = 0.0):
    """Bradley-Terry / 成对逻辑回归: -log sigmoid(r_c - r_r)。
    label_smoothing>0 时混一点反向项：偏好标注一定有误标。"""
    diff = r_chosen - r_rejected
    loss = -F.logsigmoid(diff)
    if label_smoothing > 0.0:
        loss = (1.0 - label_smoothing) * loss + label_smoothing * (-F.logsigmoid(-diff))
    return loss.mean()
```

2. 新建 `week15/day2/test_bt.py`，**四个断言自己先想**：① 与手写式 `-log σ(Δr)` 逐条一致；② margin 拉开时损失单调下降；③ 梯度符号（对 chosen 为负、对 rejected 为正）；④ margin 为 0 时损失恰好是 `ln2`。

```
import math, torch
from bt_rm import ScalarRM, bt_loss

torch.manual_seed(0)                                # 种子写死，失败才能复现
rm = ScalarRM(16)
fc, fr = torch.randn(8, 16), torch.randn(8, 16)
rc, rr = rm(fc), rm(fr)
loss = bt_loss(rc, rr)

manual = -(rc - rr).sigmoid().log().mean()
assert torch.allclose(loss, manual, atol=1e-6); print("formula OK")
assert bt_loss(rc + 5.0, rr) < loss; print("margin-monotone OK")
a = torch.tensor([2.0, -1.0], requires_grad=True)
b = torch.tensor([0.5, -3.0], requires_grad=True)
bt_loss(a, b).backward()
assert (a.grad < 0).all() and (b.grad > 0).all(); print("grad-direction OK")
assert abs(bt_loss(torch.zeros(4), torch.zeros(4)).item() - math.log(2)) < 1e-6
print("zero-margin=log2 OK")
assert bt_loss(torch.tensor([50.0]), torch.tensor([-50.0])).item() < 1e-9
print("separable->0 OK")
print("BT TESTS OK")
```

3. 跑：`cd E:/study/posttrain/week15/day2 && python test_bt.py`。
4. 加一段 NOTES：为什么用 `-logsigmoid(diff)` 而不是 `-sigmoid(diff).log()`（后者在 diff 很负时 log(0) → inf，前者是数值稳定实现）。**这个细节面试说出来是实打实的加分**。

完成判据：终端打印 5 个测试名 + `BT TESTS OK`；限时达标——从空文件到全绿 **≤35 分钟**（Day 7 闭卷复测按这个时限）；NOTES 里数值稳定性那两行写完。

> 注：一定会追问的三句，提前想好：① "BT 模型的假设是什么"——偏好只由两个回复的分数差决定，且满足独立性与传递性假设，所以它表达不了"同分但风格不同"这类偏好；② "RM 的准确率能说明什么"——held-out 偏好对准确率只说明它在**这个分布**上分得开 chosen/rejected，50% 是随机、60-70% 量级对 0.5B 是正常水位、越高越可能过拟合标注噪声；③ "DPO 和它的关系"——DPO 是把 KL 约束下的最优策略闭式解代回 BT 损失、消掉了显式 RM，所以 DPO 里那个 `beta` 就是原来 KL 系数的化身（**这条能连起来讲，八股就直接过一轮**）。

任务 3：M2 五层下挖 L1-L3（约 80 分钟）

1. 在 `interview/drill.md` 建 `## M2` 一节，先写 L1/L2/L3 三层，每层 ≥3 问，合计 **≥9 问**，每问四段（问 / 60 秒口播答 / 证据出处 / 漏洞标记）。
   - L1 例：数据多少条、哪来、清洗规则几条、lr 与 batch、adapter 参数占比、评测集几题谁判的
   - L2 例：为什么 LoRA 不选全量、为什么 1.5B 不选 0.5B、为什么 SFT 之后再 DPO、为什么基线选云端 API
   - L3 例：LoRA 省了什么代价（上限在哪）、DPO 相比 PPO 省了什么/丢了什么、为什么这个数据规模下不加 replay
2. **写不出答案的问，标** `没做过` **并进 gaps**，不许用一句空话糊过去。

完成判据：`drill.md` 的 M2 节 ≥9 问、四段齐全率 100%；`没做过` 标记 ≥2 处（**一处都没有 = 你在美化，不是在复盘**）；每问的证据栏是一个可打开的路径或"未量化"。

任务 4：投递第二批 + 台账（约 40 分钟）

1. 第二批 ≥6 家：模型公司 ≥3 + 业务侧 ≥2 + 大厂实验室 ≥1。每家同样"改首屏一句 + 3 行自述 + 登记"。
2. 台账加一列"下一步动作与日期"：`已投` 满 5 天没动静的，安排 Week 16 Day 1 统一催一次（内推码在手就有理由问进度）。

完成判据：`applications.csv` 累计 **≥22 行有投递日**（Week 13 首批 + 本周 Day 1 + 今天）；今天新增 ≥6 行；每家"下一步动作"非空。

任务 5：收工笔记（约 10 分钟）

NOTES.md：`300` 二分写法是否闭卷 AC、BT 断言里哪个第一遍没过、明天贪心预计耗时。**完成判据**：三行都写了且第一行是"是/否"结论，不是"再说说"。

今日验收：DP（下）6 行（独立 AC ≥5）+ `BT TESTS OK` 且 ≤35 分钟 + M2 下挖 ≥9 问 + 台账累计 ≥22 行。

Day 3 • 周三：贪心 6 题 + M3 讲述版 + 八股漏洞闭卷回补

今日总览（6h）：贪心与 DP 是一对，判据是"交换论证说得出来"。晚上把 M3 的 15 分钟版写完——M3 是你三个项目里**最容易被追问到崩**的一个，因为 GRPO 的曲线判据全是细节。

任务 1：贪心 6 题（约 140 分钟）

1. 题单：**55 跳跃游戏(20) / 45 跳跃游戏 II(25) / 121 买卖股票的最佳时机(15) / 122 买卖股票 II(15) / 763 划分字母区间(30) / 406 根据身高重建队列(35)**。
2. 每题写完在本子上写一句**贪心选择性**（为什么局部最优能推到全局最优，或反例在哪）。`45` 是"每一步能到的最远处"，`763` 要反向记录每个字母最后出现位置，`406` 是"按身高降序、k 值插入"。
3. 说得出每道的时间复杂度与为什么这样排序/插入。

完成判据：6 行台账，**独立 AC ≥5**；6 题的"贪心选择性"一句话全部有写（**空着的题记为未 AC，不许进累计**）。

> 注：贪心在面试里真正的考法是"你给我一个反例"。`45` 和 `122` 相对安全，`406` 一定要能讲"为什么先排高个子再按 k 插入不会被后面的矮个子弄乱"——因为后插入的人只在自己身高这一维上 counting，前面更高的人对他来说就是"已排队"的常量。讲不清就重做。

任务 2：M3 讲述版 15 分钟（约 100 分钟）

1. `interview/talk-m3.md`，六段结构与秒数分配同 Day 1 的 M2，但**"指标"段权重最高（240 秒）**：M3 的卖点不是"我跑了 GRPO"，是**我会读 RL 曲线**。
2. 必讲的三个判据（写成口播句，不是要点）：
   - 健康：reward 上行 + KL 在受控区间小幅波动

- hacking：平均输出长度暴涨而 reward 走平（配一句"我当时把 reward 拆成正确性/格式/长度三个子项来看"）
- 无信号：组内全对或全错 → 优势为零 → 题目难度分布才是 RLVR 的第一超参

3. 讲完必须落到"降级与不收敛也算成果"：如果你 M3 最终没涨，就讲"我用 1.5B + 小样本把管线跑通、把不收敛的曲线留下来做成了 fail-log 的 F-05/F-06 两条"——**这是诚实且有力的叙事，比硬吹准确率强十倍**。
4. 朗读计时两遍，目标 13-16 分钟。

完成判据：`talk-m3.md` 六段齐、"指标"段含三条曲线判据的口播句；文件头记录两遍朗读时长且落在 13-16 分钟；引用了 `fail-log.md` 里至少 2 条真实条目编号。

> 注：M3 一定会被问"你为什么不用 PPO"。标准答案不是"PPO 太难"，而是**成本结构**：PPO 要同时装 policy / value / reference / reward / rollout 五个角色，显存和优化器状态直接翻倍；GRPO 用组内基线替掉 value 网络，用规则奖励替掉 RM，单卡能跑起来。代价是它依赖"同 prompt 多个采样的可比较性"，奖励必须是可验证或可分档的——**能把代价说出来才算真懂权衡**。

任务 3：八股漏洞闭卷回补（约 60 分钟）

1. 打开 `week14/gaps.md` 里 Week 14 Day 5 闭卷答错的题 + `bagu-2.md` 所有标"半对/错"的条目，合上资料重答一遍（写进 `bagu-2.md` 末尾"第二次闭卷"区，40 分钟）。
2. 剩下 20 分钟：**口播三道经典追问**（rank 16 / reward hacking / KL 系数），各录一条 ≤90 秒，回听并按"定位术语→盯的信号→量级→双向代价→遗留"五段检查缺哪层。
3. 录音落 `week15/day3/talk/`。

完成判据：重答 14 问 **≥10 对**（比 Week 14 的口径只许升不许降）；三条录音存在、时长 ≤90 秒；每条标注"缺哪一层"。

任务 4：投递第三批 + 台账（约 40 分钟）

1. 第三批 ≥6 家，把三档都补齐（本周累计新增 ≥20 家，本周结束时台账**累计 ≥22 行**，含 Week 13 首批）。
2. 每投一家补一栏"JD 里我完全对不上的关键词"（多半是分布式训练 / 论文 / 竞赛），**统计出现次数**——这个数字是 Week 16 风险预案的判断依据。

完成判据：今天新增 ≥6 行；`applications.csv` 本周累计新增 ≥20 行；"缺口关键词"计数表写进 `targets.md` 末尾（≥5 个关键词带次数）。

任务 5：收工笔记（约 20 分钟）

NOTES.md：贪心里最别扭的一道、M3 讲述第三段是否超时、明天开卡要跑的那条命令是哪一条（**没写脚本就不许开卡**）。**完成判据**：三行齐，第三行是一整条可粘贴命令（含路径与 seed）。

今日验收：贪心 6 行（独立 AC ≥5，选择性说明齐）+ `talk-m3.md` 13-16 分钟 + 闭卷重答 ≥10/14 + 台账本周累计 ≥20 家。

Day 4 • 周四：图/层序 6 题 + 手撕 GRPO 组内优势 + M3 下挖 + 开卡补跑

今日总览（6.5h）：今天有本周唯一一次开卡，**上限 2 小时**，只为把两条"没证据"的失败案例补成有证据。手撕第 2 件写 GRPO 的组内优势——这是后训练面试里最可能被要求现场写的那 10 行。

任务 1：图/层序 6 题（约 150 分钟）

1. 题单：**695 岛屿的最大面积(25) / 130 被围绕的区域(30) / 235 二叉搜索树的最近公共祖先(20) / 199 二叉树的右视图(20) / 103 二叉树的锯齿形层序(25) / 127 单词接龙(30)**。
2. `130` 的写法是从**边界**反向 DFS 把 'O' 标走，剩下的全填 'X'（正向枚举内部块会漏）；`127` 是双向 BFS（同时从 begin/end 扩展，剪掉一半搜索量），讲得出为什么双向更快。
3. `199/103` 是 Week 14 那道 `102 层序遍历` 的两个变体：**队列长度 = 当前层节点数**这一句不变，199 只取每层最后一个、103 只加一个翻转标志——面试里能说出"三个题共用一个模板"比写对更值钱。`235` 用 BST 性质直接下降，**不许写成 236 那种通用解法**。

完成判据：6 行台账，**独立 AC ≥5**；`127` 至少写出单向 BFS，双向写法若没独立做出标 `抄会` 并进 Day 6 回炉；`199/103` 与 Week 14 的 `102` 三题共用模板的差别用一句话写在备注。

> 注：`127 单词接龙` 是 Hot 100 里少见的"建图 + BFS"综合题，也是最容易被要求优化的一道：每步生成所有替换（26×L）会爆，**改成先按通配符建桶（`*ot`、`h*t`）再扩展**，这是从 O(N·L·26) 到 O(N·L) 的关键。讲清这一步，算法面就过了。

任务 2：手撕 2 • GRPO 组内优势（约 110 分钟）

1. 新建 `week15/day4/grpo_adv.py`：

```
import numpy as np

def grpo_advantage(rewards, eps: float = 1e-6):
    """rewards: (n_prompts, G) —— 同一 prompt 采 G 个输出的规则奖励。
    返回 (adv, keep): adv 同形状，keep 标记每组是否参与更新。
    组内标准差为 0（全对/全错）时优势无信息，整组丢弃。"""
    rewards = np.asarray(rewards, dtype=np.float64)
    mean = rewards.mean(axis=1, keepdims=True)
    std = rewards.std(axis=1, keepdims=True)          # 总体标准差, ddof=0
    adv = (rewards - mean) / (std + eps)
    keep = (std.squeeze(1) > 1e-8)
    return adv * keep[:, None], keep
```

2. 新建 `week15/day4/test_grpo_adv.py`，断言自己先想：**每组均值为 0、每组标准差≈1、半对半错的两值组给出 ±1、全同组被 mask 掉且优势为 0**：

```
import numpy as np
from grpo_adv import grpo_advantage

rng = np.random.default_rng(42)                    # 种子写死，失败可复现
R = rng.normal(0.0, 1.0, size=(6, 8))
adv, keep = grpo_advantage(R)
assert np.abs(adv.mean(axis=1)).max() < 1e-12; print("zero-mean OK")
assert np.abs(adv.std(axis=1) - 1.0).max() < 1e-3; print("unit-std OK")

a, k = grpo_advantage(np.array([[1., 1., 0., 0.]]))
assert np.allclose(a[0], [1., 1., -1., -1.], atol=1e-6); print("half-split=+-1 OK")

a, k = grpo_advantage(np.array([[1., 1., 1., 1.], [0., 0., 0., 0.]]))
assert k.tolist() == [False, False] and np.all(a == 0); print("zero-var-masked OK")

a, k = grpo_advantage(np.array([[1., 0., 0., 0.]]))
assert a[0][0] > 0 and (a[0][1:] < 0).all(); print("outlier-sign OK")
print("GRPO ADV OK")
```

3. 跑：`cd E:/study/posttrain/week15/day4 && python test_grpo_adv.py`。
4. 在文件末尾注释里回答三个问题（各 ≤3 行）：① 为什么除 std 而不只减 mean（不同 prompt 的难度/奖励尺度不同，不减尺度会让简单题主导梯度）② `eps` 放分母内还是外，对零方差组行为有什么影响（**不同框架实现不同，要说清你用的是哪一种**）③ G=4 时 std 估计很粗，会有什么训练症状。

完成判据：打印 5 个断言名 + `GRPO ADV OK`；从空文件到全绿 **≤35 分钟**；三条注释写进文件（不是笔记，是代码注释——面试可能让你现场讲这段代码）。

> 注：这 10 行背后的完整叙事要提前备好：GRPO 去掉了 PPO 的 value 网络，用**组内均值当 baseline**，所以它要求同一 prompt 采 G 个输出（rollout 成本换掉了 critic 成本）；优势算完做 clip 和 KL 约束；奖励必须可验证（数学答案抽取、格式合法、规则校验）或来自 RM。追问"它和 RLOO/reinforce++ 的区别"——同样是采样基线，**GRPO 用组内标准化、RLOO 用留一（leave-one-out）均值**，能说出这一层就是准备过的。

任务 3：M3 五层下挖 L1-L5 + 两道经典追问定稿（约 90 分钟）

1. `drill.md` 的 `## M3` 一节，五层各 ≥3 问（**≥15 问**），四段齐全。L4 必放 reward hacking 与零方差组两条，L5 必放"从 1.5B 到 7B 先崩什么"（答：rollout 显存与采样吞吐，不是训练本体）。
2. 把 Day 3 录的两条（reward hacking / KL 系数）按 `drill.md` 的口径**改写成文字终稿**，五段结构一层不缺。
3. 交叉核对：讲述稿里每个数字 ↔ fail-log 条目 ↔ repo 文件，三者要能互相指认；对不上的立刻把数字降级为"量级/未量化"。

完成判据：M3 节 ≥15 问、五层全覆盖、`没做过` ≥2 处；两道经典追问文字稿标"定稿"；至少 1 个数字完成"从量级升级为带出处"或明确保留"未量化"。

任务 4：开卡补跑执行（约 40 分钟，卡时上限 2h）

1. 执行 Week 14 写好的脚本（没有就先用 10 分钟按今天的目标补一版最小脚本）：

```
ssh -p <端口> root@<你的实例地址>        # 参数在 AutoDL 控制台「复制登录指令」
cd /root/autodl-tmp && source /etc/network_turbo   # 学术加速，对 github/hf/pypi 有效
nohup bash rerun.sh > rerun.log 2>&1 &              # 挂后台跑，别开着终端等
```

2. 脚本要求（Week 14 已定，再核一遍）：显式 `cd`、模型路径写死在 `/root/autodl-tmp`、`seed` 写死、输出目录写死，跑的是**能产出那张图的最小实验**（例如 1.5B + 500 条、两个 rank 档的对照，跑完即停）。
3. 跑的时候继续做任务 1/任务 3（**挂机不算工时**）；每 20 分钟看一次：`tail -n 40 rerun.log`。
4. 收数据（把文件拉到本地再关机）：

```
scp -P <端口> root@<你的实例地址>:/root/autodl-tmp/out/rerun.log E:/study/posttrain/week15/day4/
```

5. 关机：控制台点"关机"，确认状态显示"已关机"。**任何时候不点销毁**（销毁连数据盘一起清空）。

完成判据：`week15/day4/` 下有本次日志片段或曲线截图 ≥1 个；`fail-log.md` 对应条目补上"出处：week15/day4/xxx"，数值从"未量化"升级为带出处的量级；**本周累计卡时 ≤2h、花费 ≤¥5**（看 AutoDL 计费明细，把数字记进 `review.md`）。

> 注：这一步是**买证据**，不是做实验。只跑那条支撑简历/fail-log 的最小对照，跑完就关。别顺手"再训 200 step 看看"——那是求职周最贵的三分钟。**如果开卡后环境装不起来或脚本报错 30 分钟没解决**：立刻关机（已花的算沉没），两条条目退回"未量化"，写进 gaps 推到 Week 16 的 4h 备选窗口。

今日验收：图 6 行（独立 AC ≥5）+ `GRPO ADV OK` 且 ≤35 分钟 + M3 下挖 ≥15 问 + 补跑证据 ≥1 条入库（或明确记录"退回未量化"）。

Day 5 • 周五：字符串 6 题 + 手撕 LoRA merge 与回滚 + M4 讲述版与下挖

今日总览（6h）：滑动窗口收尾，本周新题到此 30 道（累计 ≥85）。手撕第 3 件把 LoRA 的 merge/回滚闭环补上——**这是"参数高效"这个话题唯一的实战证据**。

任务 1：字符串 6 题（约 140 分钟）

1. 题单：**209 长度最小的子数组(25) / 438 找到字符串中所有字母异位词(30) / 72 编辑距离(35) / 5 最长回文子串(25) / 28 找出字符串中第一个匹配项的下标(15) / 647 回文子串(10)**。
2. `209`、`438` 是同一族滑动窗口的两个形态，写完把两个窗口收缩条件并排写在一行注释里（`209` 是"满足后尽量收缩"的变长窗、`438` 是定长窗）。`72` 是二维 DP 的地基题：三种操作各自的转移写清（替换/插入/删除对应 `dp[i-1][j-1]/dp[i-1][j]/dp[i][j-1]`），**第一行第一列初始化成 `i` 和 `j` 而不是 0，这题九成的错在这**。`28` 要真写 KMP 的 `next` 数组，写不出就退回暴力并标 `抄会`。
3. `5` 用中心扩展（奇偶两次），说清为什么比 DP 更省空间。

完成判据：6 行台账，**独立 AC ≥5**；两道滑动的收缩条件差异有一行并排注释/备注；`solved.md` 顶部写明"本周新题 30、累计独立 AC **≥85**"（Week 14 末 55 + 本周 30）。

> 注：字符串题在后训练岗位里最实际的对应是**答案抽取与格式校验**——GSM8K 的 final-answer 正则、JSON 合法性、"只认最后一个答案"。被问"你的验证器怎么写的"时，别答"用了正则"，答"先剥代码块与前后缀、再归一化数字格式（千分位/货币符/单位）、最后比对，抽取失败单独计一类错误，因为**抽取失败和模型算错是两件事**"。今天写的这些题就是那套东西的手感。

任务 2：手撕 3 • LoRA merge 与回滚（约 100 分钟）

1. 把 Week 14 的 `lora_layer.py` 复制进 `week15/day5/`，并补上 `unmerge`（面试里"merge 之后怎么回滚"的标准实现之一）：

```
    @torch.no_grad()
    def unmerge(self):
        """减回同一片 delta。float64 下可复原到 1e-9 量级；
        bf16 基座上减回的残差不可忽略 —— 所以 merge 前一定留一份未 merge 权重。"""
        self.base.weight.data -= self.scaling * (self.B @ self.A)
        self.merged = False
```

2. 新建 `week15/day5/test_merge.py`：

```
import copy, torch, torch.nn as nn
from lora_layer import LoRALinear

torch.manual_seed(7)                                    # 种子写死
base = nn.Linear(64, 32).double()
layer = LoRALinear(base, r=8, alpha=16).double()
x = torch.randn(4, 64, dtype=torch.float64)
snapshot = copy.deepcopy(base)                          # 回滚凭据：merge 前的基座副本

with torch.no_grad():
    layer.A.normal_(0, 0.05); layer.B.normal_(0, 0.1)
ref = layer(x).clone()
layer.merge()
assert torch.allclose(base(x), ref, atol=1e-5); print("merge-equiv OK")
assert layer.merged and torch.allclose(layer(x), base(x), atol=1e-12); print("no-double-path OK")

layer.unmerge()
assert torch.allclose(base.weight, snapshot.weight, atol=1e-9); print("unmerge-recover OK")
assert layer(x).dtype == ref.dtype and torch.allclose(layer(x), ref, atol=1e-5)
print("re-bypass OK")

back = LoRALinear(copy.deepcopy(snapshot), r=8, alpha=16).double()
assert torch.allclose(back(x), snapshot(x), atol=1e-12); print("reload-base-rollback OK")
n_ab = sum(p.numel() for p in layer.parameters() if p.requires_grad)
assert n_ab == 8 * (64 + 32) and n_ab < sum(p.numel() for p in snapshot.parameters())
print(f"adapter-size-ratio OK ({n_ab} vs {sum(p.numel() for p in snapshot.parameters())})")
print("MERGE TESTS OK")
```

3. 跑：`cd E:/study/posttrain/week15/day5 && python test_merge.py`。
4. 在 NOTES 里写两条服务口径：① 上线为什么要 merge（省一次旁路 matmul，推理图干净，vLLM 直接加载合并后的权重）② 什么时候**不该** merge（多任务共用一个基座、按租户热插不同 adapter、A/B 要快速回滚）。

完成判据：打印 6 个测试名 + `MERGE TESTS OK`，从空文件到全绿 **≤40 分钟**；NOTES 里两条口径各 ≥2 句；`adapter-size-ratio` 那行打印出来的两个数字抄进 NOTES（讲"adapter 只有几 MB"时要用真实量级）。

> 注：这题的杀手追问是"你 merge 过之后想换回原来的基座怎么办"。三种答法分高下：低——"重训"；中——"把 delta 减回去"（就是今天写的 `unmerge`，要能立刻说出它在低精度下留残差）；高——**"我 merge 前一定留一份未 merge 的基座副本，回滚是换权重文件而不是做算术，减 delta 只是没有副本时的补救"**。判据里的 `reload-base-rollback OK` 就是在逼你走第三种路径。

任务 3：M4 讲述版 + 五层下挖（约 90 分钟）

1. `interview/talk-m4.md` 六段成稿，M4 的卖点是**闭环**：真实产品 → 日志构造数据 → SFT/DPO/RL 三阶段 → 自建评测 + 线上 A/B → 小模型替换 API 的成本测算。
2. "指标"段必须含 A/B 协议三件：同一组 30 个真实查词场景、固定 prompt 模板、记录延迟/成本/主观分三个维度（**说清哪些是量级、哪些有表**）。
3. 朗读计时两遍（13-16 分钟），并在 `drill.md` 的 `## M4` 节写 L1-L5 各 ≥3 问（**≥15 问**，四段齐全）。
4. L5 的两问是硬指定：**"并发从 1 到 1000 先崩什么"**（答：推理吞吐与显存带宽，其次才轮到训练侧数据管道）与**"数据 ×10 先崩什么"**（答：人工质检与 judge 一致性，不是存储）。

完成判据：`talk-m4.md` 六段齐、朗读 13-16 分钟；`drill.md` M4 节 ≥15 问、五层齐、`没做过` ≥2 处；两个指定 L5 问有明确答案且不是"没想过"。

任务 4：台账跟进 + 补跑数据回收（约 30 分钟）

1. 把 `applications.csv` 全表状态刷一遍：`已投 / 笔试 / 约面 / 无回复 / 已拒`；满 5 天无动静的标"待催"，Week 16 Day 1 统一催。
2. 昨天补跑的证据挂到对应 fail-log 条目与讲述稿段落，检查引用路径能打开。

完成判据：台账状态 100% 为本周更新；`待催` 列表 ≥5 家（若一家都没有，说明投递数还不够）；至少 1 条 fail-log 的出处路径更新完成。

今日验收：字符串 6 行（独立 AC ≥5）+ 累计新题 30、`solved.md` 顶部 ≥85 + `MERGE TESTS OK` ≤40 分钟 + `talk-m4.md` 成稿 + M4 下挖 ≥15 问。

Day 6 • 周六：回炉 + 第 3 次模拟面（对镜录屏）+ 三份讲稿定稿

今日总览（6.5h）：本周大考。模拟面按**对镜录屏**执行，全程不暂停；下午按今天暴露的问题把三份讲稿定稿。

任务 1：未 AC 与"抄会"题回炉（约 80 分钟）

1. 从 `solved.md` / `log.md` 筛出本周标 `抄会` / `未AC` 的题，闭卷重敲，每题限时 25 分钟。
2. 通过的改标"独立 AC（隔日重做）"，仍不过的写进 `gaps.md` 并排到 Week 16 的复习位。

完成判据：回炉 ≥5 题且**重做通过 ≥4 题**；`log.md` 状态同步；DP 类若有一道仍不过，Week 16 Day 1 必须优先它。

任务 2：第 3 次模拟面 • 对镜录屏（约 110 分钟，含前置 10 分钟）

1. 形式：**对镜录屏**（手机架在电脑前，屏幕共享你的编辑器）。本周不找人互面——第 3 次的目的不是测知识，是**测三件事**：六段讲述节奏、被打断后能否回到主线、手撕的限时完成度。前置 10 分钟只摆放设备与抽题。
2. 抽题用脚本，种子写死（复盘时能还原"我当时抽到哪几问"）。**先建题库**：`drill.md` 到这里已有三项目 ≥45 问，把每问的"问"那一行单独抽出来存成 `interview/drill_questions.txt`（一行一问）。抽法取决于你在 `drill.md` 里怎么标每问——如果前面统一带了 `Q:` 或 `**问：**` 就一条 grep 搞定；**没统一格式就手工抽，45 行约 15 分钟，别为省时间现改正则**（Week 16 会继续用这个文件，一次弄好）。

```
cd E:/study/llm-posttraining-lab/interview
grep -oP '(?<=^\*\*问：\*\*).*' drill.md > drill_questions.txt   # 或 grep '^Q:' drill.md | cut -c4-
wc -l drill_questions.txt    # 期望 ≥45；明显偏少说明格式没对上
```

```
# week15/day6/draw.py
import random
random.seed(20251220)                          # 写死，保证可复现
bag = [l.strip() for l in open('drill_questions.txt', encoding='utf-8') if l.strip()]
for q in random.sample(bag, 8):
    print(q)
```

3. 结构（计时执行，总时长约 92 分钟，不许暂停重来）：
   - 自我介绍 60 秒（2 分钟）
   - M2 讲述 15 分钟 + 追问 5 问（10 分钟）
   - M3 讲述 15 分钟 + 追问 5 问（10 分钟，**必含 rank 16 / reward hacking / KL 系数三问**）
   - 手撕 20 分钟：从 BT 损失与 GRPO 组内优势里抽一题，**空文件起写、不查资料、运行调试 ≤2 次**
   - LeetCode 一道 Medium（15 分钟）：从本周没重做过的 `抄会` 题里抽
   - 反问 5 分钟（对着镜头把反问清单念一遍，练口吻）

完成判据：录屏文件存在（文件名 `mock-03-<日期>`，时长 **≥85 分钟**，落 `week15/day6/`）；六个环节全部执行到（缺任一段在复盘里记"未完整"，Week 16 总模拟面补上）；手撕那 20 分钟有明确的"写完/没写完"结论。

任务 3：模拟面复盘（约 80 分钟）

1. 回看，按 Week 13 定下的三列表逐题写：`题/问 | 我答了什么（尽量原话）| 正确答案或更优答法 | 下次怎么说（口播 ≤80 字）`，落 `interview/mock-03.md`。
2. 过程指标三条，**与 mock-02 同一口径对照**：① 被打断次数 ② "嗯…就是…"式空转总时长 ③ 说"我记不清了/没测过"的次数（这个数不是越低越好，而是**要出现在该出现的地方**——真没测过的问题必须说没测过）。
3. 每条问题标"改哪个文件"：讲稿某段 / drill 某问 / fail-log 某条 / 简历首屏。

完成判据：`mock-03.md` 三列复盘 **≥12 条**；三项过程指标有数字且与 mock-02 有对照（写明升/降）；≥3 条落到具体文件的修改动作，且当天改完。

> 注：第 3 次模拟面最常见的崩点是**讲述被打断后接不回来**。练法很具体：被打断 → 用一句话回应对方 → 立刻说"我回到刚才那页"并复述你上一段的结论句。这个动作在 `talk-*.md` 每段开头写一句"结论句"，就是为了让打断有锚点。**没有结论句的讲稿，被打断一次就废。**

任务 4：三份讲稿 + drill 定稿（约 90 分钟）

1. 按复盘的修改动作改 `talk-m2/m3/m4.md` 与 `drill.md`，每份文件头写"定稿版本 + 日期"。
2. 三份讲稿各录一条 **3 分钟压缩版**（面试里"简单讲讲这个项目"用的是这个），落 `interview/talk-3min/`。
3. 最终核对数字纪律：所有对外数字要么有 repo 内出处，要么写"量级/约"，要么写"未量化"。**精确假数字一处都不许留**。

完成判据：三份讲稿与 drill 全部标"定稿"；drill 三项目合计 **≥45 问**、五层覆盖无空层；三条 3 分钟录音存在且各 ≤3 分 20 秒；抽查 3 个数字都能指到出处或明确降级。

任务 5：简历口播复录 + 周账（约 30 分钟）

简历 60 秒版复录一遍（对照 Week 14 的那条，要能听出进步）；`review.md` 记本周实际用时（对照 42h）与实际卡时（对照 2h / ¥5 上限）。

完成判据：录音存在且 ≤75 秒；`review.md` 两行账写完（含 AutoDL 计费明细截图或数字）。

今日验收：回炉 5 题（≥4 过）+ `mock-03` 录屏 ≥85 分钟 + 复盘 ≥12 条（含与 mock-02 对照）+ 三份讲稿定稿 + drill ≥45 问。

Day 7 • 周日：补漏 + 闭卷复测 + 总验收 + Week16 预习

今日总览（4.5h）：不排新内容。清洞、闭卷复测三题、勾总验收表，最后 60 分钟预习 Week 16（总模拟面 + 收口 + 全期资产盘点）。

任务 1：补漏（约 60 分钟）

打开 `gaps.md`（本周新增 + Week 13/14 未清的洞）。**规则：最多补 2 个洞**。优先级：`drill.md` 里标 `没做过` 但你判断一定会被问的 > 闭卷答错的八股 > 未 AC 的题 > 讲述超时的段落。

完成判据：补的 2 个洞在 gaps 里标"已清 + 清法"；其余每个洞都写"推到哪周、用什么动作推"（**不许留"以后再说"**）。

> 注：Day 7 不排新内容不是奖励，是为了把"看了"和"会了"分开。补漏与复测必须同日完成；总验收表里凡填不出数字的项一律算没通过，写"基本完成"等于没写——这张表 Week 16 要拿去算 16 周的总账。

任务 2：闭卷复测 3 题（约 50 分钟）

只看题面、合上所有资料：

1. 手撕抽一题重敲（BT 损失或 GRPO 组内优势，**≤20 分钟**，跑到 `BT TESTS OK` / `GRPO ADV OK`）。
2. 纸上写清三件事：DPO 里的 `beta` 与 RM/PPO 里 KL 系数的关系与区别；GRPO 替掉了 PPO 的哪个网络、代价是什么；健康曲线与 hacking 曲线各两种判据。
3. 抽 M4 的一个项目段口播 3 分钟压缩版（不看稿），录音 ≤3 分 20 秒。

完成判据：三题全完成；第 1 题 20 分钟内全绿；第 2 题三条各 ≥3 句且概念不混；第 3 题录音存在且时长达标。

任务 3：本周总验收（约 30 分钟）

逐项勾选，结果写进 `week15/review.md`：

#	验收项	判据
1	刷题	本周新题 ≥30（独立 AC ≥25）；`solved.md` 累计独立 AC **≥85**；`log.md` 与它行数差 ≤3
2	手撕 ML	BT 损失 / GRPO 组内优势 / LoRA merge 三件全绿（`BT TESTS OK`、`GRPO ADV OK`、`MERGE TESTS OK`），限时分别 ≤35/≤35/≤40 分钟
3	讲述版	`talk-m2/m3/m4.md` 三份 15 分钟版成稿，朗读 13-16 分钟；三条 3 分钟压缩版录音存在
4	五层下挖	`drill.md` 三项目合计 ≥45 问、L1-L5 无空层；三道经典追问"好答案"五段结构齐全并定稿
5	投递	`applications.csv` 本周新增 ≥20 家、累计 ≥22 行有投递日；内推 ≥8 家；缺口关键词计数表已成文
6	模拟面	第 3 次模拟面录屏 ≥85 分钟、六环节执行；`mock-03.md` 复盘 ≥12 条且含过程指标对照
7	周账与资产	卡时 ≤2h、花费 ≤¥5 有计费依据；补跑证据入库或明确退回"未量化"；fail-log 与讲稿数字互相指认一致

≥6 项通过 = 本周合格；不足 6 项，把缺的写进 Week 16 Day 1 的前 2 小时。**完成判据**：7 项里 ≥6 项打勾，`review.md` 每一项后面填了实际数字（题数、分钟数、家数、条目数），不许留空或写"基本完成"。

任务 4：Week 16 预习与 gaps 交接（约 60 分钟）

1. 读 Week 16 的 Day 1-2（总模拟面结构 + 手撕 4 题限时），先把**反问清单**的 10 个问题草稿写进 `interview/questions-to-ask.md`。
2. 把 16 周资产清单开个头：`M1 / M2 / M3 / M4 + 6 篇博客 + 数据集 + 简历 + 投递数`，每行写"在哪、能不能点开给面试官看"（Week 16 Day 7 要在这张表上收口）。
3. gaps 交接：`drill.md` 里 `没做过` 的条目 → Week 16 Day 5 的行为面/反问准备；未 AC 题 → Week 16 Day 1 复习位；`待催` 投递 → Week 16 Day 1 上午统一催一次；卡时没做成 → Week 16 的 4h 备选窗口。

完成判据：反问草稿 ≥10 条；资产清单 ≥8 行且每行有路径；gaps 每条都有目标周与动作。

任务 5：缓冲（约 70 分钟）

前面顺就提前收工。不顺就用这一小时补 `drill.md` 最空的层（多半是 L5），或把 Day 6 没写完的手撕补完。**完成判据**：结束时 `drill.md` 的 L5 层三项目各 ≥3 问（或 Day 6 手撕限时内重跑全绿），二者至少完成一个并记录是哪一个。

今日验收：`review.md` 验收表 ≥6 项打勾 + 复测三题达标 + gaps 全部有目标周。

三、常见问题与纪律

- **某天崩了怎么办？** 不熬夜追。未完成项按「天-任务号」写进 `gaps.md`，Day 7 前 1 小时与缓冲就是给它们的。连续两天崩，砍 Day 3 任务 3（八股回补）和 Day 5 任务 4（台账跟进），**保刷题、三份讲稿 + drill、集中投递、模拟面**。
- **投递石沉大海怎么办？** 这是常态不是信号。唯一正确的动作是继续按日计划走 + 把 JD 里你对不上的关键词计数（Day 3 任务 4 那一栏）。**同一个要求连续被 ≥5 家拒**才叫信号——多半是"分布式训练"或"论文/竞赛"，处理办法见总计划风险 3：横移投数据构造 / 评测 / Agent 后训练岗（技能重叠约八成、竞争小得多），进大厂再转组是常规路径。不许因为没回复就回去改简历第 8 版或重跑实验。
- **单题/单点卡多久放手？** 刷题按今天限时（15-40 分钟），到点看题解只看到思路、第二天重做才计数。非刷题卡点 40 分钟无解就写 NOTES + gaps 跳过；**开卡卡点例外：30 分钟没跑起来就关机**，已花的钱算沉没。
- **卡时与钱**：本周上限 2h、预估 ¥3-5。开卡前 `rerun.sh` 必须已存在；跑完即关机，确认"已关机"；**任何时候不点销毁**。计费明细截图进 `review.md`。
- **模拟面不是测试**：它唯一作用是暴露"讲不出"。第 3 次要盯的还是**被打断后的恢复能力、空转时长、有没有结论句**，不是通过率。
- **数字纪律（本周最容易破功）**：讲稿与 drill 里的每个数字要么有出处路径，要么写"量级/约"，要么写"未量化"。补跑没做成也不许回填精确值——面试官追问三个细节就露馅，一露馅全盘皆输。
- **AI 工具的边界**：允许它解释报错、给算法思路方向、review 你写完的代码、帮你把口播稿改顺；**不允许**替你 AC 题、替你写项目经历、替你答五层下挖。现场手撕与追问时你身边没有它。